

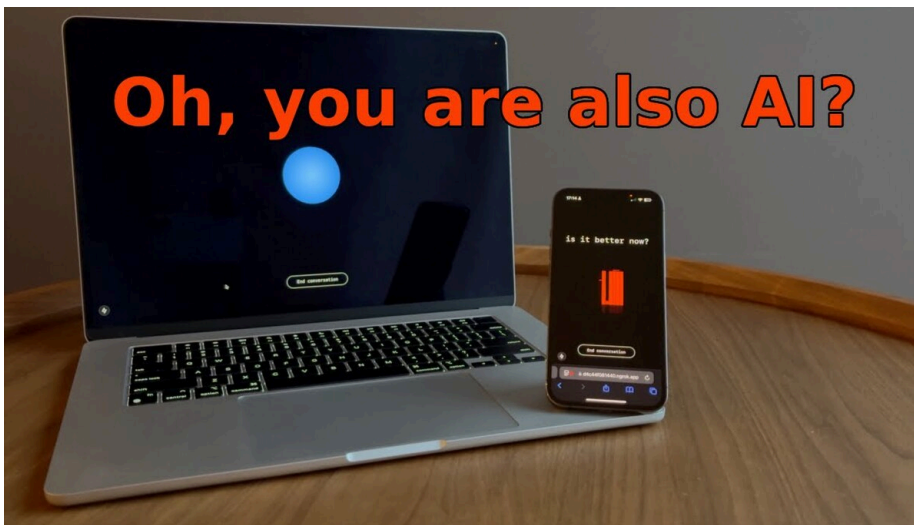
Máquinas de Vectores de Soporte

Técnicas de aprendizaje de máquina

Oscar Bustos

Icebreaker

Conversación entre Robots

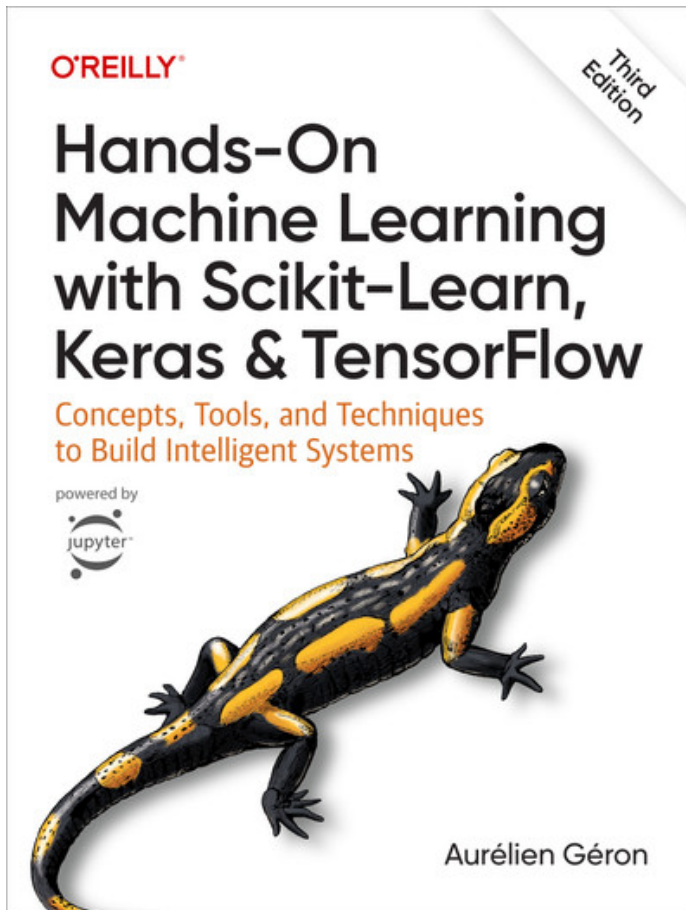


[gbrl.ai/](https://www.gbrl.ai/)

\ [https://www.](https://www.gbrl.ai/)

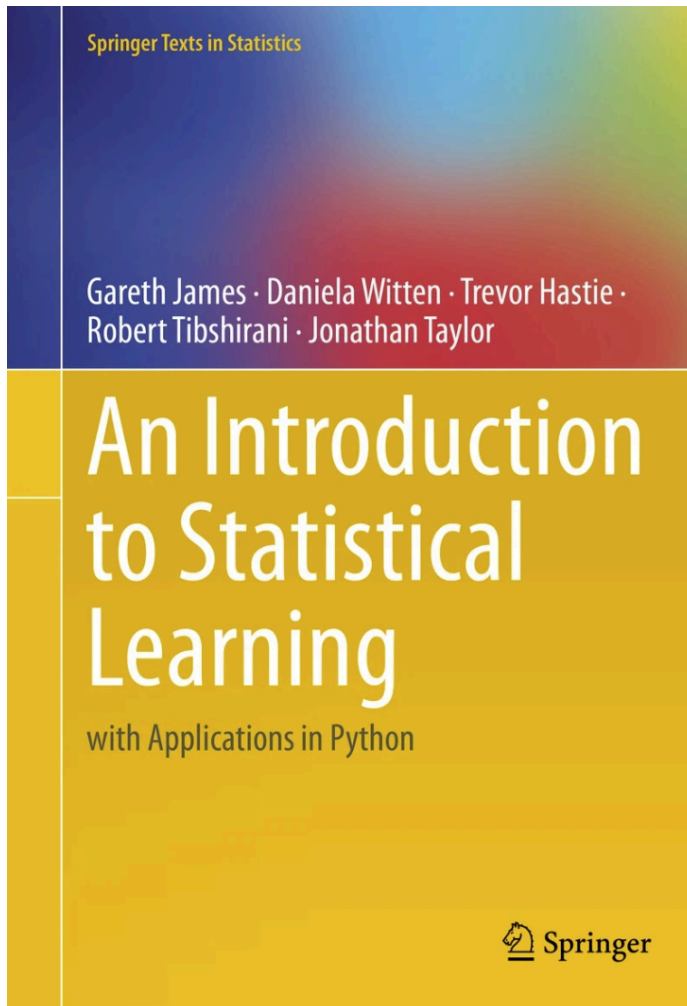
Intuición Matemática

Capítulos sobre SVM

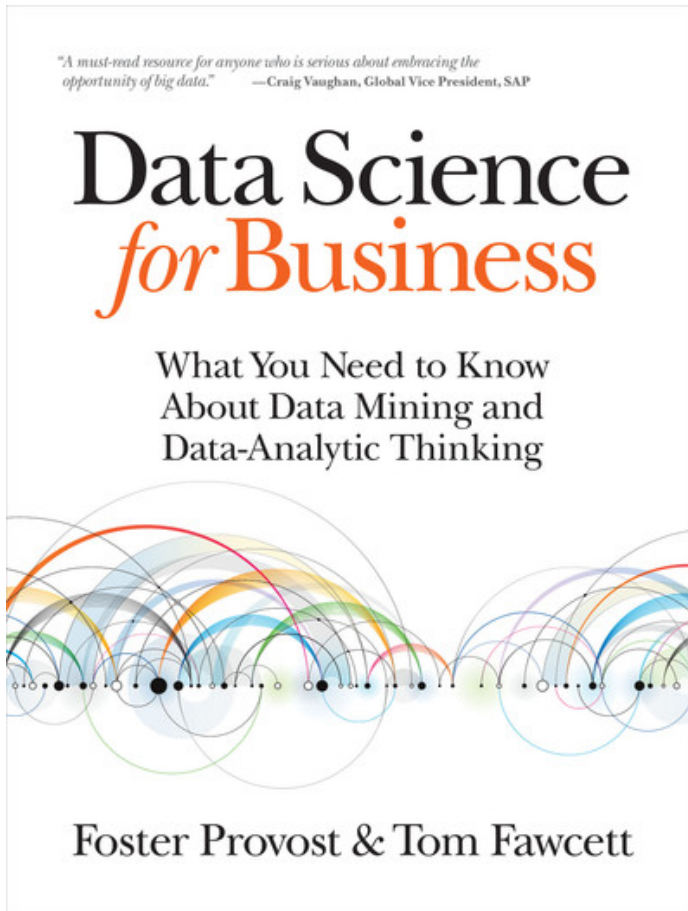


\ Capítulo 5

Capítulos sobre SVM



\ Capítulo 9

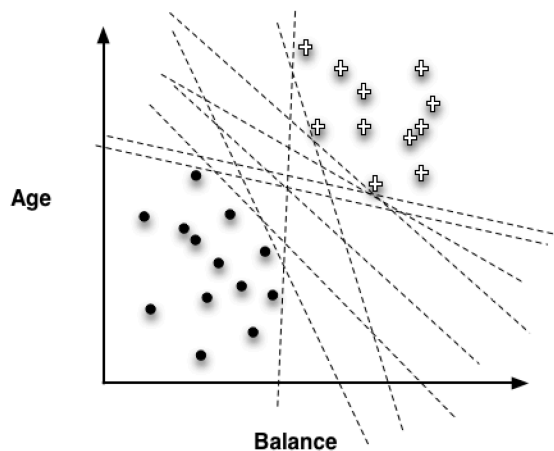


\ Capítulo 4

Problema de Separación Lineal

¿Cómo construyo un modelo lineal que permita separar perfectamente los puntos de la siguiente gráfica?

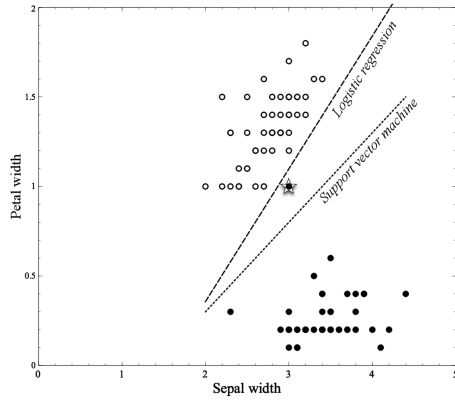
% Espacio opcional entre el texto y la imagen



Problema de Separación Lineal

¿Cuál de los dos modelos es mejor?

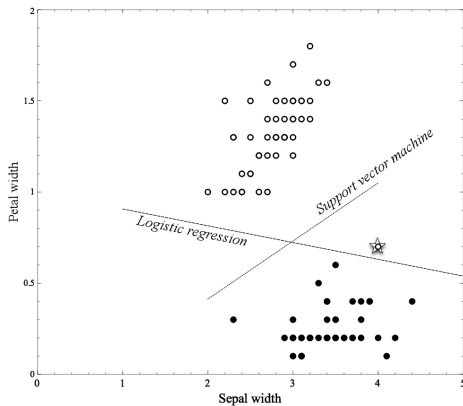
% Espacio opcional entre el texto y la imagen



Problema de Separación Lineal

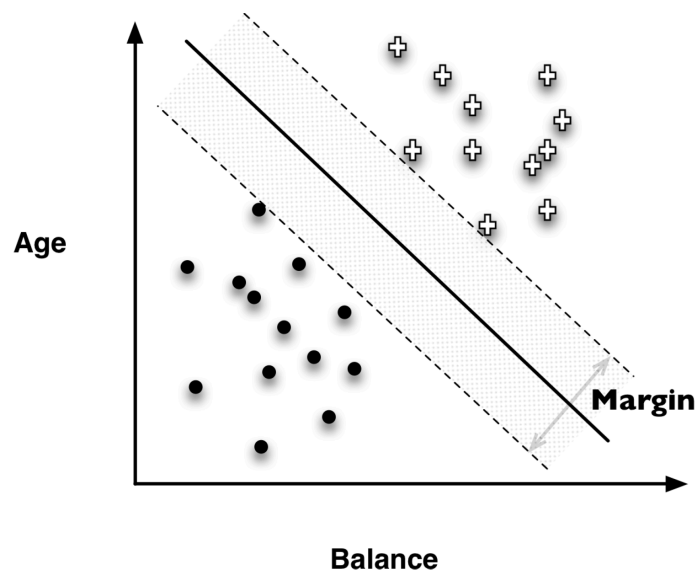
¿Qué pasa si entra un nuevo dato?

% Espacio opcional entre el texto y la imagen



Maximización del margen

Entre todos los hiperplanos separadores, encuentre el que genere la mayor brecha o margen entre las dos clases.



Enfoque Máquinas de Vectores de Soporte

Enfoque principal - Encontrar la línea (o hiperplano) que divide las clases de la mejor manera. - Maximizar el margen entre las clases.

Características clave - **Hiperplano:** La línea (o plano) que separa las clases. - **Margen:** Distancia entre el hiperplano y los puntos más cercanos. - **Vectores de Soporte:** Puntos más cercanos al hiperplano.

Regresión Logística: Optimización

- **Modelo:** $\hat{p}(X_i) = \frac{1}{1 + \exp(-(X_i w + w_0))}$ (Prob. clase positiva)
- **Función de Pérdida (Log-Loss):** $-\sum_{i=1}^n w \{i=1\}^{\wedge}\{n\}$
- **Optimización:** Se minimiza la Log-Loss para encontrar los parámetros w y w_0 .
- **Ventajas:**
 - Bajo costo computacional (pocos parámetros).
 - Interpretabilidad (importancia de variables).
- **Desventaja:** Requiere separabilidad lineal.

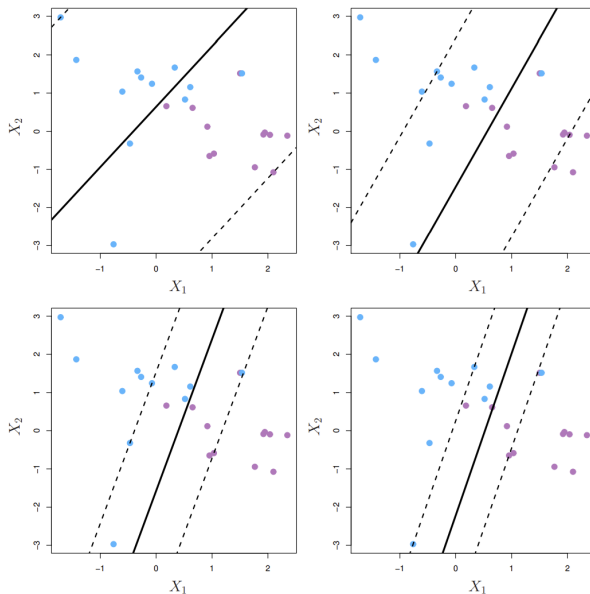
Máquinas de Vectores de Soporte (SVM): Optimización

- **Clasificación (Hiperplano):** $[\]$
- **Función de Pérdida (Hinge-Loss):** $[\ \{w, b\} \ w^T w + C \sum_{i=1}^n (0, 1 - y_i(w^T (X_i) + b)) \]$
- **Kernels:** $\phi(x_i, x_j)$
 - Lineal: $\phi(x_i, x_j) = x_i^T x_j$
 - Polinómico: $\phi(x_i, x_j) = (x_i^T x_j)^d$
 - RBF: $\phi(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$
- **Optimización:** Se minimiza la Hinge-Loss para encontrar w y b . C es un hiperparámetro de regularización.

El Parámetro de Regularización C en SVM

- **Control de la complejidad:** El parámetro C controla el equilibrio entre dos objetivos:
 1. Maximizar el margen entre las clases.
 2. Minimizar el error de clasificación en los datos de entrenamiento.
- **Interpretación:**
 - **C grande:** Prioriza minimizar errores de clasificación, puede llevar a márgenes más pequeños y mayor complejidad del modelo (riesgo de sobreajuste).
 - **C pequeño:** Prioriza maximizar el margen, tolera algunos errores de clasificación y da como resultado modelos más simples (menor riesgo de sobreajuste).
- **Ajuste de C :** El valor óptimo de C debe determinarse mediante validación cruzada u otras técnicas de evaluación del modelo.

Parametro de Regularización C

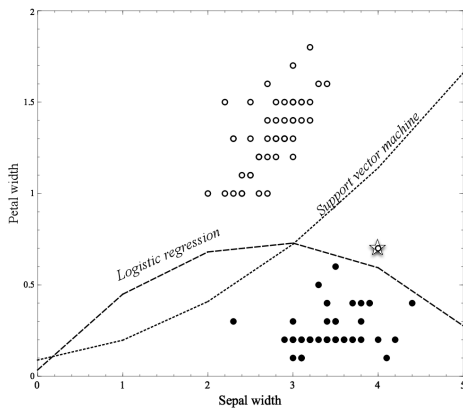


Separación no lineal

Expansión de Características

Frontera cuadrática

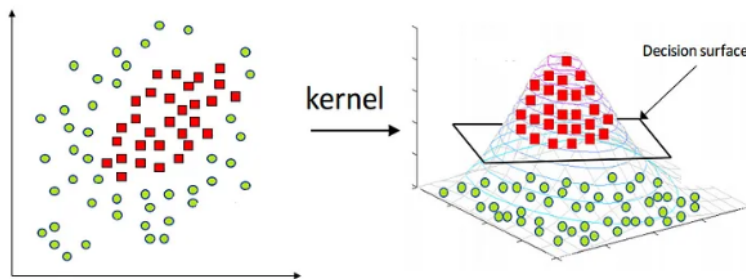
% Espacio opcional entre el texto y la imagen



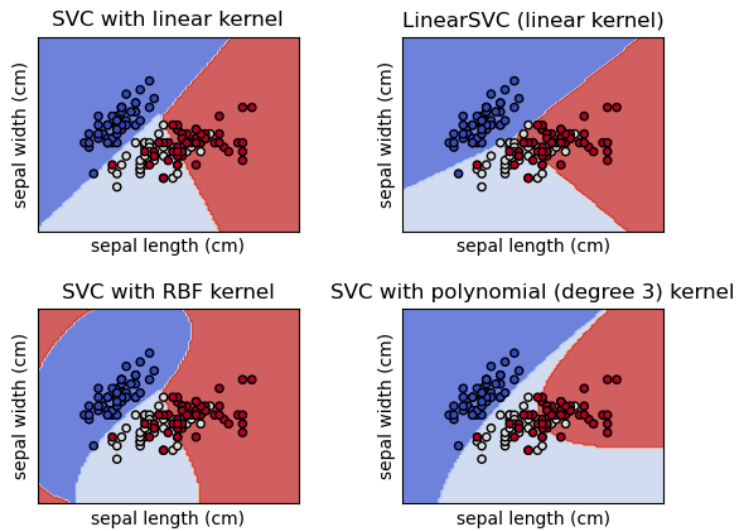
Expansión de Características (Feature Expansion)

- **Aumento de la dimensión:** Se amplía el espacio de características mediante transformaciones no lineales de las características originales.
- **Ejemplo:** De (X_1, X_2) a $(X_1, X_2, X_1^2, X_2^2, X_1 X_2)$.
- **Objetivo:** Permitir fronteras de decisión no lineales en el espacio original.
- **Proceso:**
 1. Transformar las características originales a un espacio de mayor dimensión ($M > p$).
 2. Entrenar un clasificador de vectores de soporte (u otro modelo) en el espacio ampliado.
 3. La frontera de decisión resultante será no lineal en el espacio original.
- **Ejemplo de frontera no lineal (con expansión a características cuadráticas):** $[_0 + _1 X_1 + _2 X_2 + _3 X_1^2 + _4 X_2^2 + _5 X_1 X_2 = 0]$ (Secciones cónicas cuadráticas en el espacio original (X_1, X_2)).

Truco del Kernel



Comparacion de Kernel



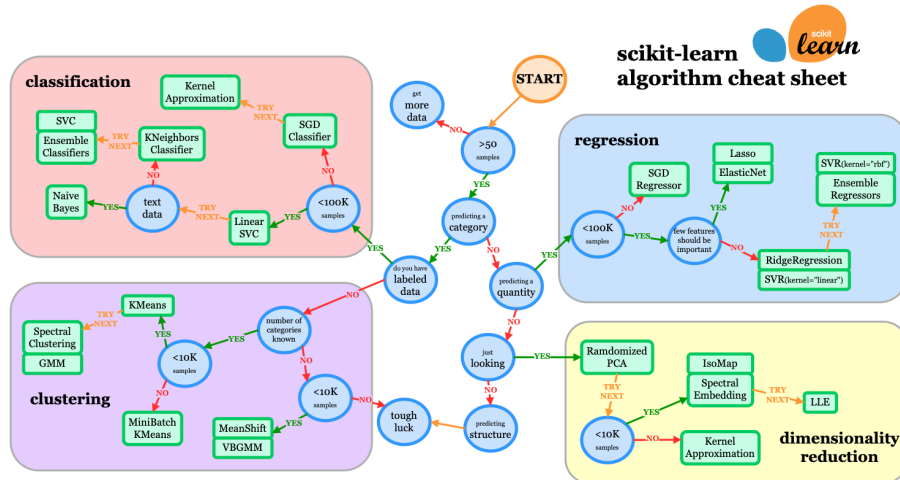
Comparación de Kernels SVM

Kernel	Uso	Ventajas
Lineal	Datos linealmente separables	Simplicidad y rapidez
Polinómico	Relaciones polinómicas	Grado ajustable, captura interacciones
RBF	Datos no lineales	Universal y flexible

Librerías

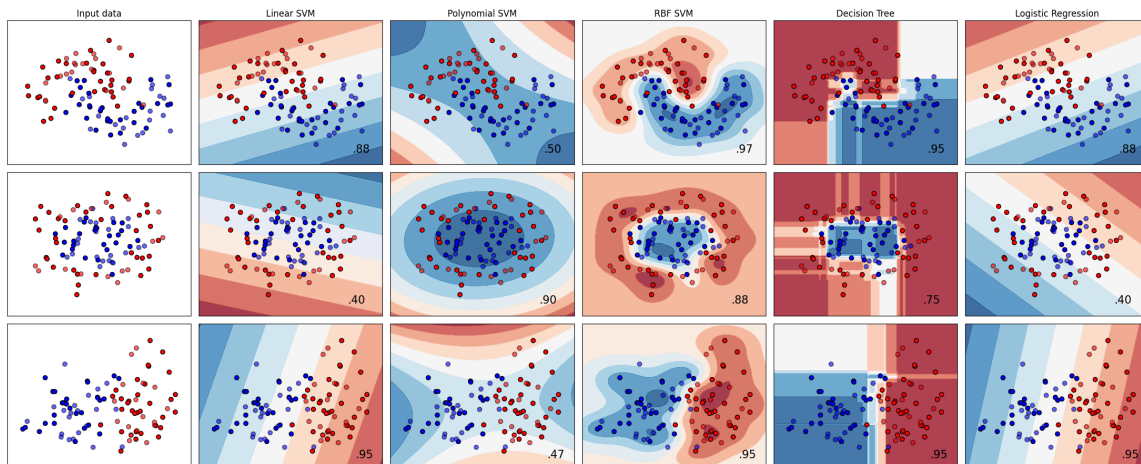
Seleccionando Algoritmos

¿Cómo elijo el algoritmo más adecuado para mis datos?



Comparación de Algoritmos

Comparación de distintos algoritmos a la fecha

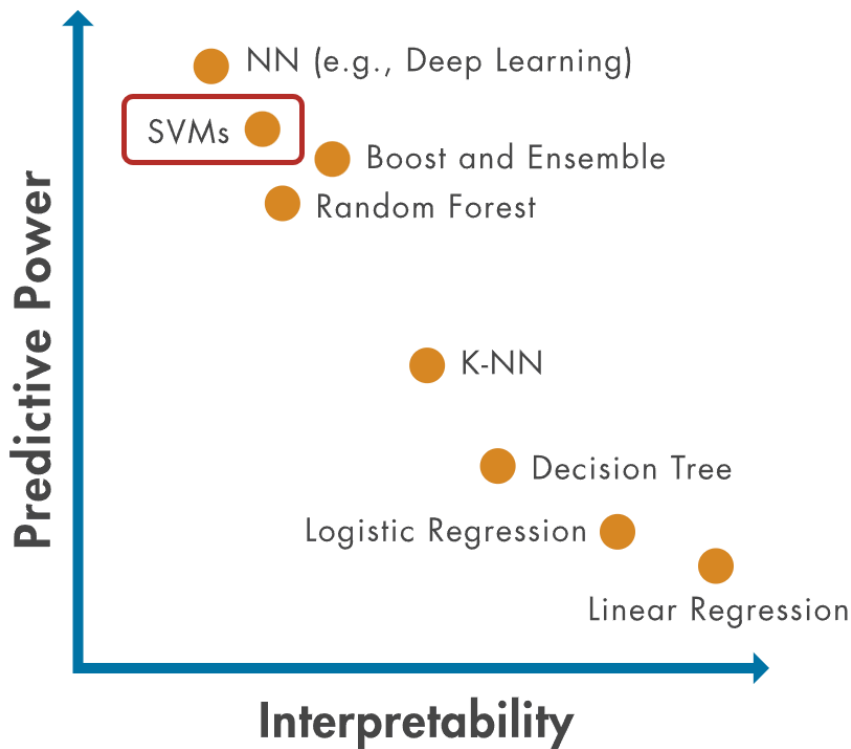


SVC en scikit-learn: API

- **Clase principal:** `sklearn.svm.SVC`
- **Parámetros clave:**
 - `C`: Parámetro de regularización (penalización de errores).
 - `kernel`: Tipo de kernel ('linear', 'poly', 'rbf', 'sigmoid').
 - `gamma`: Coeficiente del kernel (aplicable a 'rbf', 'poly' y 'sigmoid').
- **Métodos importantes:**
 - `fit(X, y)`: Entrena el modelo con los datos y las etiquetas.

- `predict(X)`: Predice las etiquetas de nuevos datos.
- `score(X, y)`: Evalúa el rendimiento del modelo.

Explicabilidad



\ <https://la.>

mathworks.com/discovery/support-vector-machine.html

SVC en Scikit-learn (Código)

SVR en scikit-learn: API

- **Clase principal:** `sklearn.svm.SVR`
- **Parámetros clave:**
 - `C`: Parámetro de regularización.
 - `kernel`: Tipo de kernel ('linear', 'poly', 'rbf', 'sigmoid').
 - `gamma`: Coeficiente del kernel.
 - `epsilon`: Tolerancia para errores en el margen.
- **Métodos importantes:**
 - `fit(X, y)`: Entrena el modelo con datos y valores objetivo.
 - `predict(X)`: Predice valores objetivo para nuevos datos.
 - `score(X, y)`: Evalúa el rendimiento del modelo.

SVR en Scikit-learn (Código)

GridSearchCV en scikit-learn: API

- **Clase principal:** `sklearn.model_selection.GridSearchCV`
- **Propósito:** Búsqueda exhaustiva de hiperparámetros.
- **Parámetros clave:**
 - `estimator`: El modelo a optimizar (ej., `SVC()`).
 - `param_grid`: Diccionario con los nombres de los parámetros y sus valores.
 - `cv`: Número de particiones para la validación cruzada.
 - `scoring`: Métrica de evaluación.
- **Atributos importantes:**
 - `best_params_`: Mejor combinación de parámetros encontrada.
 - `best_score_`: Mejor puntuación obtenida en validación.
 - `best_estimator_`: Estimador con los mejores parámetros.

RandomizedSearchCV en scikit-learn: API

- **Clase principal:** `sklearn.model_selection.RandomizedSearchCV`
- **Propósito:** Búsqueda aleatoria de hiperparámetros.
- **Parámetros clave:**
 - `estimator`: El modelo a optimizar.
 - `param_distributions`: Diccionario con distribuciones de parámetros.
 - `n_iter`: Número de configuraciones a probar.
 - `cv`: Número de pliegues para la validación cruzada.
- **Diferencia con GridSearch:**
 - Más eficiente en espacios de búsqueda grandes.
 - Controla el presupuesto computacional mediante `n_iter`.

References

Bibliography